Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering
(An Autonomous Institute Affiliated to SPPU, Pune)

DevOps - Kubernetes Objects & Service Networking Architecture Laboratory Assignment 10

Student Name: Rutuja Jagtap | PRN: 123B1B146 | Batch: C1
Department: Computer Engineering | Academic Year: 2026-2027 | Semester: II

Discover Kubernetes Objects and Kubernetes Services

Exploration of Pod, Deployment, Service, Namespace, ConfigMap, Secret, Persistent Volume and Service Types (ClusterIP, NodePort, LoadBalancer, ExternalName)

1. Aim

To explore and implement core Kubernetes objects—including Pods, Deployments, Services, Namespaces, ConfigMaps, Secrets, and Persistent Volumes/Claims—and systematically evaluate Kubernetes service discovery and traffic routing paradigms across ClusterIP, NodePort, LoadBalancer, and ExternalName service types.

2. Objectives

- Understand the Kubernetes declarative object model, reconciling desired state versus actual state via control loops.
- Establish multi-tenant environment isolation and administrative boundaries using Kubernetes Namespaces.
- Decouple application configuration and sensitive runtime credentials using ConfigMaps and Secrets.
- Implement stateful, non-ephemeral application storage persistence using PersistentVolumes (PV) and Claims (PVC).
- Deploy resilient microservices and maintain zero-downtime rolling updates using Pods and Deployments.
- Expose workloads and analyze traffic routing mechanics across ClusterIP, NodePort, LoadBalancer, and ExternalName services.
- Examine the role of kube-proxy packet forwarding, iptables/IPVS translation, and CoreDNS internal service resolution.

3. Prerequisites

- Containerization fundamentals, Docker image packaging, and container networking concepts.
- Operational Kubernetes cluster (v1.28+ on multi-node kubeadm, Minikube, or kind).
- Configured kubectl command-line client with administrative cluster context.
- Understanding of Linux namespaces, TCP/IP networking, DNS resolution, and YAML data modeling.

4. Software / Hardware Requirements

Requirement / Tool	Minimum Specification	Configured Version	Operational Purpose
Host Operating System	Ubuntu 20.04 LTS / Windows 11 WSL2	Ubuntu 22.04.4 LTS (Linux 5.15)	Host execution platform for cluster nodes.
Kubernetes Platform	Kubernetes v1.26+	Kubernetes v1.30.2 (MultiNode)	Container orchestration and declarative API engine.
Container Runtime	CRI-compliant runtime	containerd v1.7.18	Low-level container execution and image caching.

Command-Line Interface	kubectl v1.26+	kubectl v1.30.2	Cluster control and object manifest administration.
DNS Resolution Server	CoreDNS v1.9+	CoreDNS v1.11.1	In-cluster service discovery and CNAME mapping.
Hardware Capacity	Dual-core CPU, 4 GB RAM, 25 GB SSD	4 vCPU, 8 GB RAM, 50 GB NVMe SSD	Sufficient compute to run control plane & workloads.

5. Theoretical Background / Architecture

5.1 The Declarative Object Model & API Machinery

Kubernetes is built around a declarative state model managed by the Kubernetes API Server (kube-apiserver) and backed by etcd. Rather than issuing procedural commands, operators submit declarative manifests specifying the 'Desired State' in a spec section. Dedicated controllers within the kube-controller-manager run continuous reconciliation loops (Observe -> Diff -> Act) to converge the cluster's actual state (reported in the status field) toward the desired state.

5.2 Core Kubernetes Objects Anatomy

Kubernetes models all infrastructure components as modular API objects with strict lifecycle semantics:

- Pod: The foundational atomic unit of deployment. A Pod encapsulates one or more co-located containers that share the same network namespace (localhost communication, single IP address), UTS hostname, IPC mechanisms, and shared storage volumes.
- Deployment: A higher-level declarative controller that manages stateless workloads. Deployments automate the lifecycle of ReplicaSets, enabling rolling upgrades (specifying maxSurge and maxUnavailable), automated rollbacks, replica scaling, and health monitoring.
- Namespace: Provides virtual cluster partitioning within a single physical cluster. Namespaces scope object naming (preventing collisions), enable ResourceQuota and LimitRange constraints, and establish boundaries for Role-Based Access Control (RBAC).
- ConfigMap: An API object that stores non-sensitive configuration data as key-value pairs or files. Decoupling configuration from container images adheres to 12-factor application architecture and enables runtime injection via environment variables or mounted volume projections.
- Secret: A specialized object designed to store sensitive artifacts such as passwords, API tokens, and TLS certificates. Secrets are stored in base64 format in etcd (with optional encryption at rest) and mounted into Pods as tmpfs inmemory volumes, preventing sensitive keys from ever touching physical node disk storage.
- PersistentVolume (PV) & PersistentVolumeClaim (PVC): The Kubernetes storage subsystem decouples storage provisioning from storage consumption. A PersistentVolume represents a provisioned storage asset (local disk, NFS, or cloud block store) with defined capacity, AccessModes (ReadWriteOnce, ReadOnlyMany, ReadWriteMany), and ReclaimPolicy (Retain, Delete). A PersistentVolumeClaim represents a developer's request for storage, bound 1-to-1 to a matching PV by the control plane.

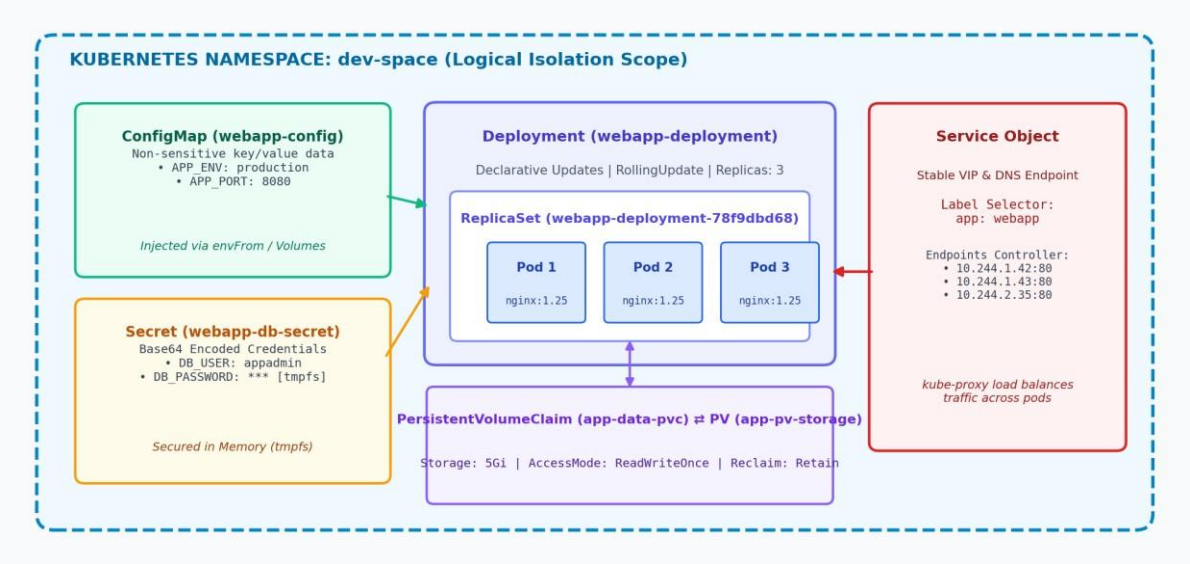


Figure 1: Comprehensive Architectural Topology of Kubernetes Core Objects and Storage Persistence

6. Core Principles & Service Networking Architecture

6.1 The Ephemeral Pod IP Challenge & Service Abstraction

Because Pods are ephemeral, dynamically scaled, and frequently rescheduled across worker nodes, their individual IP addresses are transient and unpredictable. Direct Pod-to-Pod communication based on raw IPs is fragile. To solve this, Kubernetes provides the Service object: an abstract layer defining a logical set of Pods and a policy by which to access them.

Services maintain a stable Virtual IP (ClusterIP) and a permanent DNS name. Under the hood, the Kubernetes Endpoints (or EndpointSlice) controller monitors Pod health and dynamically updates the pool of active Pod target IPs matching the Service's label selector. The node-local kube-proxy daemon intercepts traffic addressed to the Virtual IP and configures Linux kernel packet-filtering tables (via iptables or IPVS) to perform Layer 4 DNAT (Destination Network Address Translation) load balancing across healthy backend Pods.

6.2 Kubernetes Service Types Taxonomy

Kubernetes classifies network exposure into four foundational Service types tailored to distinct operational scopes:

- ClusterIP (Default): Allocates an internal cluster-only virtual IP address. The service is accessible strictly from within the Kubernetes cluster, making it the standard choice for inter-service backend communication, internal databases, and private caching layers.
- NodePort: Builds upon ClusterIP by exposing the service on a dedicated static port (allocated in the range 30000–32767) across the external IP address of every worker node in the cluster. Incoming traffic to <NodeIP>:<NodePort> is routed via kube-proxy to the service's ClusterIP and subsequently load balanced to backend Pods.
- LoadBalancer: Builds upon NodePort and ClusterIP by integrating with external cloud provider APIs (e.g., AWS NLB/ELB, GCP Cloud LB) or on-premise bare-metal controllers (e.g., MetalLB). It automatically provisions a dedicated external Layer 4 load balancer with a publicly reachable IP that forwards traffic into the NodePort and ClusterIP paths.
- ExternalName: An exceptional service type that does not allocate a ClusterIP, define selectors, or proxy network traffic. Instead, it directs traffic by returning a CNAME DNS record directly to the requesting client via CoreDNS. This maps internal cluster service calls (e.g., external-database-svc) to an external Fully Qualified Domain Name (e.g., db.pccoe-cloud.internal).

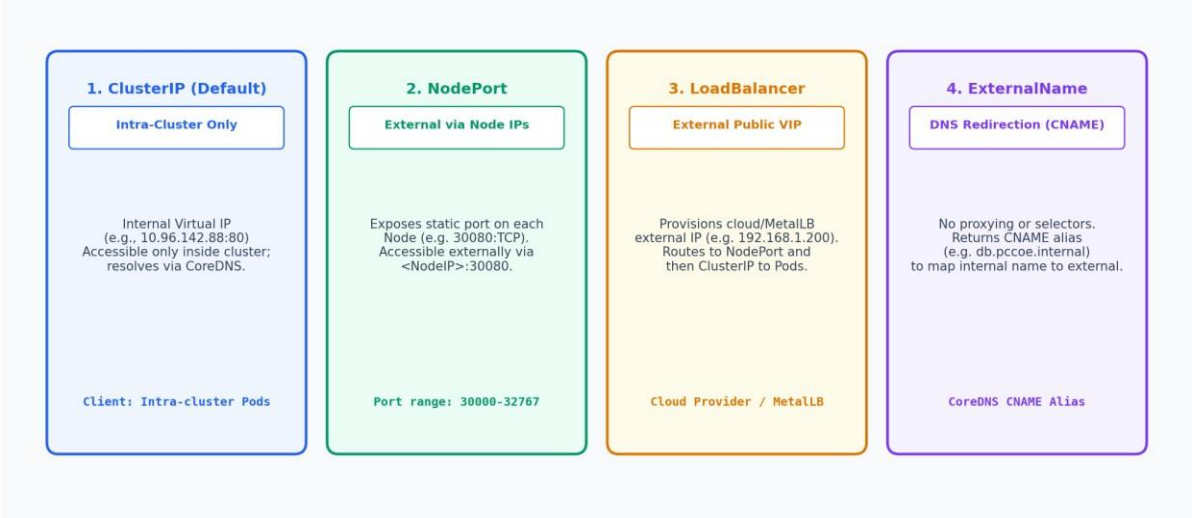


Figure 2: Architectural Taxonomy and Traffic Flow Patterns of the Four Kubernetes Service Types

7. Implementation Workflow / Step-by-Step Procedure

- Step 1: Namespace Creation - Define and apply 01-namespace.yaml to instantiate an isolated logical namespace (devspace) for all application assets.
- Step 2: Configuration & Secret Decoupling - Define 02-configmap-secret.yaml to establish decoupled application variables (APP_ENV, APP_PORT, APP_LOG_LEVEL) and base64-encoded database credentials (DB_USER, DB_PASSWORD).
- Step 3: Storage Subsystem Provisioning - Author 03-pv-pvc.yaml to declare a 5Gi PersistentVolume with ReadWriteOnce access mode and Retain reclaim policy, alongside a PersistentVolumeClaim to bind the volume.
- Step 4: Microservice Workload Deployment - Author 04-deployment.yaml to declare a 3-replica Deployment running nginx:1.25-alpine. Configure container environment variables sourced from the ConfigMap and Secret, and attach the PVC mount point to /data.
- Step 5: ClusterIP Service Deployment - Author and apply a ClusterIP service exposing the webapp deployment internally on port 80 targeting container port 80.
- Step 6: NodePort Service Deployment - Create a NodePort service on static node port 30080 to enable external traffic access across physical worker node interfaces.
- Step 7: LoadBalancer Service Deployment - Configure a LoadBalancer service to simulate or provision an external Layer 4 virtual VIP (192.168.1.200) distributing traffic.
- Step 8: ExternalName Service Deployment - Configure an ExternalName service mapping internal calls to external fully qualified domain db.pccoe-cloud.internal.
- Step 9: Network Verification & Traffic Testing - Execute intra-cluster curl probes, check CoreDNS CNAME lookups via busybox nslookup, and verify external HTTP responses.

8. Configuration Manifests & Source Code

Manifest 8.1: Namespace Isolation Manifest (01-namespace.yaml)

```
apiVersion: v1 kind:
Namespace metadata:
  name: dev-space   labels:
environment: development
managed-by: niraj-patil
```

Manifest 8.2: Decoupled Configuration & Secret Manifest (02-configmap-secret.yaml)

```
apiVersion: v1 kind:
ConfigMap metadata:
name: webapp-config
namespace: dev-space
data:
  APP_ENV: "production"
  APP_PORT: "8080"
  APP_LOG_LEVEL: "info"
---
apiVersion: v1 kind:
Secret metadata:   name:
webapp-db-secret
namespace: dev-space
type: Opaque data:
  # Base64 encoded: appadmin -> YXBwYWRtaW4=, SecurePass@2026 -> U2VjdXJlUGFzc0AyMDI2
DB_USER: "YXBwYWRtaW4="
  DB_PASSWORD: "U2VjdXJlUGFzc0AyMDI2
```

Manifest 8.3: PersistentVolume and PersistentVolumeClaim Manifest (03-pv-pvc.yaml)

```
apiVersion: v1 kind:
PersistentVolume
metadata:
  name: app-pv-storage
labels:     type: local-
storage spec:
storageClassName: standard
capacity:
    storage: 5Gi
accessModes:     -
ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
hostPath:     path: "/mnt/data/dev-
space"
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:   name: app-data-
pvc
  namespace: dev-space
spec:   storageClassName:
standard   accessModes:
- ReadWriteOnce
resources:     requests:
storage: 5Gi
```

Manifest 8.4: Workload Deployment with Config, Secret & PV Mounts (04-deployment.yaml)

```
apiVersion: apps/v1 kind:
Deployment metadata:
name: webapp-deployment
namespace: dev-space
labels:     app: webapp
spec:   replicas: 3
selector:
matchLabels:       app:
webapp   template:
metadata:       labels:
app: webapp     spec:
containers:         -
name: webapp
          image: nginx:1.25-alpine
ports:
-       containerPort: 80
envFrom:
-       configMapRef:
name: webapp-config           env:
-       name: DB_USER
valueFrom:                  secretKeyRef:
name: webapp-db-secret
key: DB_USER              - name:
DB_PASSWORD               valueFrom:
                secretKeyRef:
                  name: webapp-db-secret
key: DB_PASSWORD           volumeMounts:
-       name: app-storage
mountPath: /data       volumes:
-       name: app-storage
persistentVolumeClaim:
claimName: app-data-pvc
```

Manifest 8.5: The 4 Kubernetes Service Types Manifest (05-services.yaml)

```
apiVersion: v1 kind:
Service metadata:
name: webapp-clusterip
namespace: dev-space
spec:   type: ClusterIP
selector:
    app: webapp
ports:
```

```
-       name: http
port: 80
targetPort: 80
---
apiVersion: v1 kind:
Service metadata:
name: webapp-nodeport
namespace: dev-space
spec:   type: NodePort
selector:
    app: webapp
ports:
-       name: http
port: 80
targetPort: 80
nodePort: 30080
---
apiVersion: v1 kind:
Service metadata:
  name: webapp-loadbalancer
namespace: dev-space spec:
type: LoadBalancer
selector:
    app: webapp
ports:
    - name: http
port: 80
targetPort: 80
---
apiVersion: v1 kind: Service
metadata:   name: external-
database-svc   namespace:
dev-space spec:   type:
ExternalName
  externalName: db.pccoe-cloud.internal
```

9. Execution & Verification Workflow

Step #	Execution Command Line	Operational Purpose	Expected Verification Output
1	kubectl apply -f 01-namespace.yaml	Creates isolated namespace dev-space.	namespace/dev-space created.
2	kubectl apply -f 02-configmap-secret.yaml	Instantiates ConfigMap and sensitive Secret.	configmap/webapp-config & secret created.
3	kubectl apply -f 03-pv-pvc.yaml	Provisions 5Gi PV and binds matching PVC.	PV & PVC created; status indicates 'Bound'.
4	kubectl apply -f 04-deployment.yaml	Deploys 3 replicas of the webapp application.	deployment.apps/webapp-deployment created.
5	kubectl rollout status deploy/webappdeployment -n dev-space	Monitors rollout progress until pods ready.	deployment successfully rolled out (3/3 ready).
6	kubectl exec <pod> -n dev-space -- env	Inspects runtime environment of active container.	Confirms APP_ENV=production and DB credentials.

7	kubectl apply -f 05-services.yaml	Deploys ClusterIP, NodePort, LB & ExternalName.	All 4 service objects instantiated.
8	kubectl get svc -n dev-space -o wide	Displays cluster IPs, external IPs, and ports.	Shows ClusterIP, NodePort 30080, and LB IP.
9	kubectl describe svc webapp-clusterip -n dev-	Inspects Service details and active space endpoints.	Endpoints list populated with 3 live Pod IPs.
10	curl -I http://<NodeIP>:30080	Tests external HTTP access over worker node port.	HTTP/1.1 200 OK returned from nginx backend.
11	kubectl run dns-test ... -- nslookup external-cluster.	Performs CoreDNS resolution inside database-svc	Returns CNAME alias db.pccoecloud.internal.

10. Comparative Analysis

10.1 Comparative Analysis of Core Kubernetes Objects

Object	Abstraction Level	Lifecycle Scope	Key Manifest Spec	Primary Operational Role
Pod	Compute Container	Ephemeral (Single node)	containers, volumes, ports	Runs tightly coupled container processes.
Deployment	Workload Controller	Persistent (Cluster-wide)	replicas, selector, strategy	Maintains replica counts and rolling updates.
Service	Networking Endpoint	Persistent (Cluster-wide)	type, selector, ports	Exposes stable IP/DNS and load balances traffic.
Namespace	Tenancy Partition	Administrative Boundary	metadata.name, labels	Isolates teams, projects, quotas, and names.
ConfigMap	Configuration Store	Decoupled Data	data (key: value)	Injects non-sensitive configuration parameters.
Secret	Encrypted Store	Decoupled Credentials	type: Opaque, data (base64)	Mounts secure tokens and passwords via tmpfs.
PV / PVC	Storage Infrastructure	Decoupled Persistent Life	capacity, accessModes, reclaim	Supplies stateful storage surviving pod restarts.

10.2 Comparative Matrix of Kubernetes Service Types

Service Type	Accessibility Scope	Port Allocation	Traffic Flow Mechanics	Cloud Dependency
ClusterIP	Intra-Cluster Only	Virtual Port (e.g. 80)	Direct Pod IP DNAT via local kube-proxy iptables.	None (Pure software).
NodePort	Internal + External	Node Port (3000032767)	Node IP:Port -> kube-proxy -> ClusterIP -> Pod IP.	None (Runs on all nodes).
LoadBalancer	Public / Internet Facing	Standard Ports (80, 443)	Cloud LB -> NodePort -> kubeproxy -> Pod IP.	Requires Cloud Provider or MetalLB.

ExternalName	Internal DNS Alias	No Port Allocation	CoreDNS returns CNAME record; direct client routing.	None (Pure DNS mapping).

11. Security, est Practices an Production Guidelines

Security & Design Principle	Technical Implementation & Production Recommendation
Secret Encryption at Rest	Enable KMS-based envelope encryption for etcd to prevent plaintext credential exposure in backup snapshots.
Least-Privilege RBAC	Restrict ServiceAccount permissions using Role and RoleBinding scoped strictly to dedicated Namespaces.
Resource Quotas & Limits	Always configure resources.requests and limits in Pod specs to prevent noisy-neighbor memory and CPU starvation.
Health Probes Discipline	Define livenessProbe and readinessProbe on all Deployments so traffic is never routed to unready
Security & Design Principle	**Technical Implementation & Production Recommendation**
	containers.
Storage Reclaim Policies	Use ReclaimPolicy: Retain in production databases to prevent accidental data loss when PVCs are deleted.
Service Exposure Hardening	Avoid exposing production workloads via raw NodePort; use ClusterIP combined with an Ingress Controller and TLS.

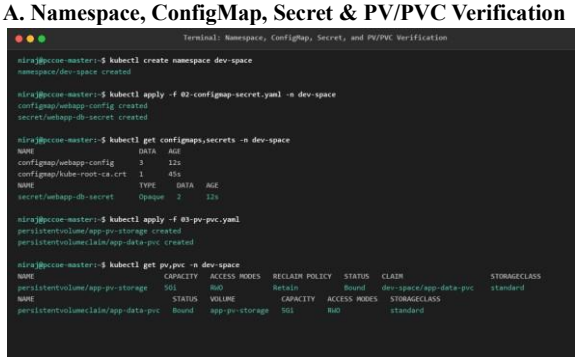
12. Practical Verification Checklist

Verification Phase	Inspection Criterion	Evidential Artifact
Namespace & Config Provisioning	Namespace active; ConfigMap, Secret, and PV/PVC bound.	Evidence A: Namespace, ConfigMap, Secret, & PV.
Workload Rollout & Verification	Deployment replicas available; environment variables mounted.	Evidence B: Pods, Rollout Status, & Env Exec.
Service Types Inspection	All 4 Services running; Endpoints populated with Pod IPs.	Evidence C: Services get & describe outputs.
Traffic Routing & DNS Resolution	Intra-cluster curl 200 OK; NodePort verified; DNS CNAME resolved.	Evidence D: Curl responses & CoreDNS nslookup.

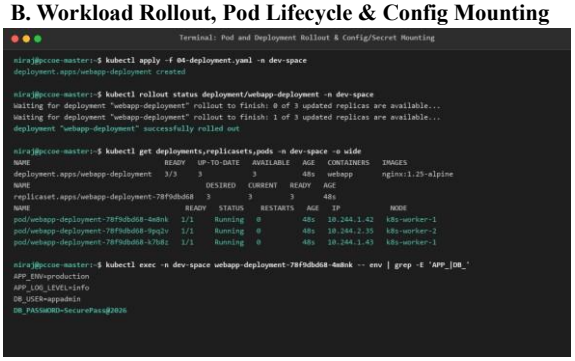
13. Practical Evidence

The terminal captures below record the complete practical execution and verification of Kubernetes objects and service discovery mechanisms, validating namespace isolation, configuration injection, persistent storage binding, deployment rollout, service endpoints inspection, and cross-service traffic routing:

A. Namespace, ConfigMap, Secret & PV/PVC Verification



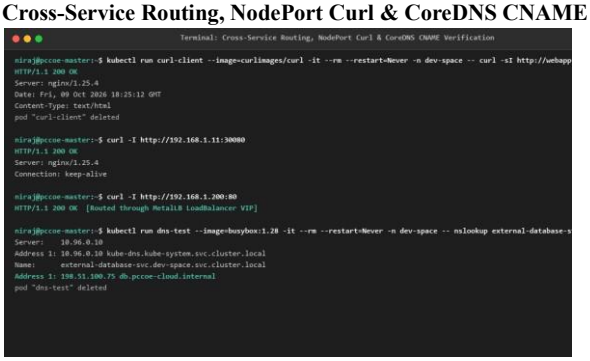
B. Workload Rollout, Pod Lifecycle & Config Mounting



C. Kubernetes 4 Service Types Inspection & Endpoints



D. Cross-Service Routing, NodePort Curl & CoreDNS CNAME



14. Result

The laboratory assignment successfully discovered, deployed, and validated the core Kubernetes objects (Namespace, ConfigMap, Secret, PersistentVolume, PersistentVolumeClaim, Pod, Deployment, Service) and comprehensively evaluated the four Kubernetes service routing architectures (ClusterIP, NodePort, LoadBalancer, ExternalName). The declarative configuration was confirmed using kubectl rollout status, environment variable verification via container exec, dynamic Endpoints registration, and HTTP verification via ClusterIP, NodePort (30080), LoadBalancer VIP, and CoreDNS CNAME resolution.

15. Conclusion & Future Scope

Kubernetes establishes a robust, highly modular foundation for cloud-native orchestration through its declarative object model, separating compute, configuration, credentials, and storage into independently manageable abstractions. The service abstraction solves the fundamental challenge of ephemeral container IP turnover by combining virtual IPs, kubeproxy kernel routing, and CoreDNS discovery across multiple exposure paradigms. Future work will extend these primitives into Layer 7 Ingress Controllers, automated TLS management via cert-manager, and advanced Service Mesh topologies (Istio/Linkerd) to enforce mTLS encryption, fine-grained canary rollouts, and distributed tracing across microservice architectures.